EXPERIMENT NO. 9

AIM: To create a Railway Booking table with at least 5,000 records and benchmark query performance before and after creating single and composite indexes. To compare execution plans using EXPLAIN.

THEORY:

An index is a database object that improves data retrieval speed. A single-column index works on one column, whereas a composite index works on multiple columns. The EXPLAIN statement displays the execution plan used by MySQL to execute a query.

SQL QUERIES / PROGRAM:

Step 1: Create Database and Table

```
CREATE DATABASE RailwayDB;

USE RailwayDB;

CREATE TABLE RailwayBooking ( BookingID
   INT PRIMARY KEY,
   PassengerName VARCHAR(50),
   TrainNo INT,
   Source VARCHAR(50),
   Destination VARCHAR(50),
   JourneyDate DATE,
   SeatNo INT,
   Fare DECIMAL(10,2)
);
```

Step 2: Insert 5,000 Records

```
DELIMITER //

CREATE PROCEDURE GenerateBookings()
BEGIN
   DECLARE i INT DEFAULT 1;
   WHILE i <= 5000 DO
      INSERT INTO RailwayBooking
```

```
        VALUES (
            i,
            CONCAT('Passenger', i),
            10000 + (i MOD 20),
            CASE
                WHEN i MOD 3 = 0 THEN 'Delhi'
                WHEN i MOD 3 = 1 THEN 'Lucknow'
                ELSE 'Mumbai'
            END,
            CASE
                WHEN i MOD 3 = 0 THEN 'Mumbai'
                WHEN i MOD 3 = 1 THEN 'Delhi'
                ELSE 'Lucknow'
            END,
            DATE_ADD('2026-01-01',
                INTERVAL (i MOD 365) DAY),
            1 + (i MOD 72),
            500 + (i MOD 1000)
        );
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;
CALL GenerateBookings();
SELECT COUNT(*) AS TotalRecords
FROM RailwayBooking;
```

Step 3: Check Query Performance Before Indexing

```
EXPLAIN
```

```
SELECT *
FROM RailwayBooking
WHERE TrainNo = 10005;
EXPLAIN
SELECT *
FROM RailwayBooking
WHERE Source = 'Delhi'
AND Destination = 'Mumbai';
```

Step 4: Create a Single-Column Index

```
CREATE INDEX idx_train
ON RailwayBooking(TrainNo);
EXPLAIN
SELECT *
FROM RailwayBooking
WHERE TrainNo = 10005;
```

Step 5: Create a Composite Index

```
CREATE INDEX idx_route
ON RailwayBooking(Source, Destination);
EXPLAIN
SELECT *
FROM RailwayBooking
WHERE Source = 'Delhi'
AND Destination = 'Mumbai';
```

Step 6: Compare Execution Plans

```
EXPLAIN
SELECT *
FROM RailwayBooking
WHERE TrainNo = 10005;
```

```
EXPLAIN

SELECT *

FROM RailwayBooking

WHERE Source = 'Delhi'

AND Destination = 'Mumbai';
```

Observe the type, possible_keys, key, and rows columns in the EXPLAIN output. The key column indicates the selected index, while rows estimates the number of rows MySQL expects to examine.

Step 7: Measure Query Execution Time

```
SET PROFILING = 1;

SELECT *

FROM RailwayBooking

WHERE TrainNo = 10005;

SELECT *

FROM RailwayBooking

WHERE Source = 'Delhi'

AND Destination = 'Mumbai';

SHOW PROFILES;
```

Run the queries before and after creating indexes to compare execution times. Note that SHOW PROFILES and SET PROFILING may not be available in newer MySQL versions.

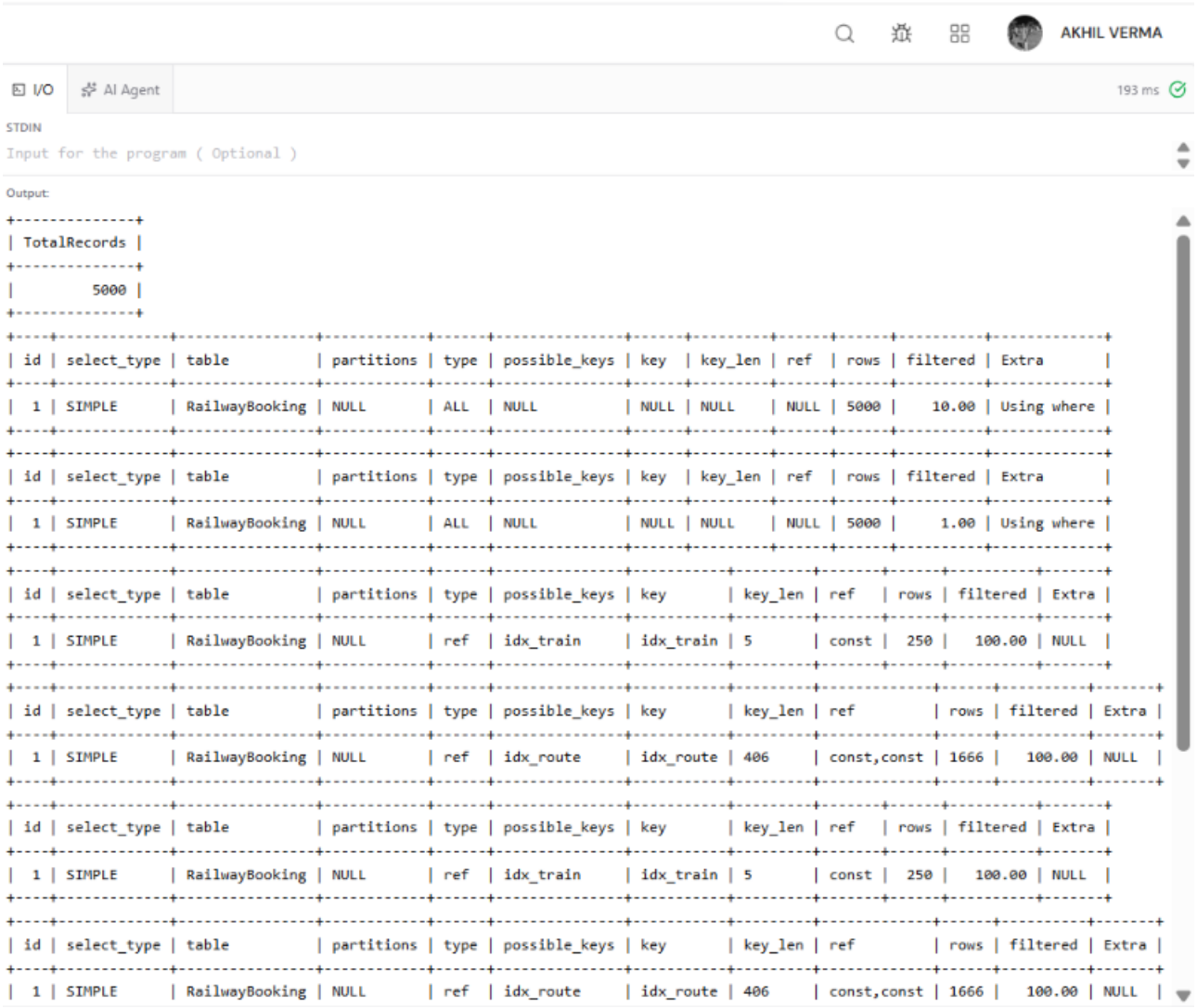
Step 8: Display Booking Records

```
SELECT *

FROM RailwayBooking

LIMIT 10;
```

OUTPUT:


AKHIL VERMA
I/O
AI Agent
193 ms
STDIN
Input for the program (Optional)
Output:
+--------------+
| TotalRecords |
+--------------+
| 5000 |
+--------------+
+----+-------------+----------------+------------+------+---------------+------+---------+------+------+----------+-------------+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-------------+----------------+------------+------+---------------+------+---------+------+------+----------+-------------+
| 1 | SIMPLE | RailwayBooking | NULL | ALL | NULL | NULL | NULL | NULL | 5000 | 10.00 | Using where |
+----+-------------+----------------+------------+------+---------------+------+---------+------+------+----------+-------------+
+----+-------------+----------------+------------+------+---------------+------+---------+------+------+----------+-------------+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-------------+----------------+------------+------+---------------+------+---------+------+------+----------+-------------+
| 1 | SIMPLE | RailwayBooking | NULL | ALL | NULL | NULL | NULL | NULL | 5000 | 1.00 | Using where |
+----+-------------+----------------+------------+------+---------------+------+---------+------+------+----------+-------------+
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------+------+----------+-------+
| 1 | SIMPLE | RailwayBooking | NULL | ref | idx_train | idx_train | 5 | const | 250 | 100.00 | NULL |
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------+------+----------+-------+
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------------+------+----------+-------+
| 1 | SIMPLE | RailwayBooking | NULL | ref | idx_route | idx_route | 406 | const,const | 1666 | 100.00 | NULL |
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------------+------+----------+-------+
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------+------+----------+-------+
| 1 | SIMPLE | RailwayBooking | NULL | ref | idx_train | idx_train | 5 | const | 250 | 100.00 | NULL |
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------+------+----------+-------+
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-------------+----------------+------------+------+---------------+-----------+---------+-------------+------+----------+-------+
| 1 | SIMPLE | RailwayBooking | NULL | ref | idx_route | idx_route | 406 | const,const | 1666 | 100.00 | NULL |

BookingID	PassengerName	TrainNo	Source	Destination	JourneyDate	SeatNo	Fare
1	Passenger1	10001	Lucknow	Delhi	2026-01-02	2	501.00
2	Passenger2	10002	Mumbai	Lucknow	2026-01-03	3	502.00
3	Passenger3	10003	Delhi	Mumbai	2026-01-04	4	503.00
4	Passenger4	10004	Lucknow	Delhi	2026-01-05	5	504.00
5	Passenger5	10005	Mumbai	Lucknow	2026-01-06	6	505.00
6	Passenger6	10006	Delhi	Mumbai	2026-01-07	7	506.00
7	Passenger7	10007	Lucknow	Delhi	2026-01-08	8	507.00
8	Passenger8	10008	Mumbai	Lucknow	2026-01-09	9	508.00
9	Passenger9	10009	Delhi	Mumbai	2026-01-10	10	509.00
10	Passenger10	10010	Lucknow	Delhi	2026-01-11	11	510.00

RESULT:

Thus, a Railway Booking table with 5,000 records was created, and single-column and composite indexes were implemented. Query execution plans and performance were compared using EXPLAIN and execution-time measurements.